SOLAR▸SWARM

Technical Reference
Martin Pham | Perth, AU

Agent Reference

Each agent is a Python module in `agents/`. All agents follow a common pattern: read from SQLite, call GPT-4o (or use a rule-based fallback), write results back to SQLite, and log to the cold ledger.

master_agent.py — The General (Tier 1)

Purpose: Strategic command. Generates experiment ideas, scores them, red-teams them, routes them, and allocates budget.

Scheduler trigger: Every 6 hours via `IntervalTrigger(hours=6)`

Key functions:

FUNCTION	PARAMS	RETURNS
`run()`	—	`list` of experiment dicts created this cycle
`_generate_ideas()`	—	`list` of 3 raw idea dicts from GPT-4o
`_process_idea(idea_data)`	`idea_data: dict`	Scored/routed experiment dict or `None`
`_mock_ideas()`	—	3 hardcoded mock ideas (used when no OpenAI key)

What it reads: `experiments` (active count, budget), `pheromone_signals` (via warm memory), `cold_ledger` (past learnings)

What it writes: `experiments` (new rows with status `approved` or `pending`)

Example output:

```
{
  "experiment_id": 42,
  "idea_text": "Cold email sequence to 50 solar SMEs in WA...",
  "confidence_score": 7.3,
  "devil_score": 4,
  "status": "approved",
  "bucket": "explore",
  "budget_aud": 37.50
}
```

Halts when: `is_halted()` returns True (circuit breaker Red state).

research_agent.py — Research Head (Tier 2)

Purpose: Manages the research task queue. Routes `research_*` job types to the appropriate handler, posts pheromone signals on completion, and queues routine market analysis if the queue is empty.

Scheduler trigger: Daily 09:00 UTC (as part of `_run_department_heads`)

Key functions:

FUNCTION	PARAMS	RETURNS
`run(max_tasks=5)`	`max_tasks: int`	`{tasks_processed, tasks_failed, signals_posted}`
`queue_prospect(company_name, suburb)`	`str, str`	`task_id: int`
`_dispatch_task(job_type, context)`	`str, dict`	Result dict
`_prospect_company(context)`	`{company_name, suburb}`	Research dict + pheromone data
`_analyse_market(context)`	`{vertical, keywords}`	Market analysis dict
`_analyse_competitor(context)`	`{competitor_name}`	Competitive intel dict

Job types handled: `research_prospect`, `research_market`, `research_competitor`

What it reads: `task_queue` (tier=3, job_type LIKE 'research_%')

What it writes: `task_queue` (status updates), `pheromone_signals` (via `post_pheromone`), `cold_ledger`

---

content_agent.py — Content Head (Tier 2)

Purpose: Manages the content task queue. Generates ad copy, email sequences, LinkedIn posts, and SMS templates via GPT-4o with Australian B2B copywriting style.

Scheduler trigger: Daily 09:00 UTC (as part of `_run_department_heads`)

Key functions:

FUNCTION	PARAMS	RETURNS
`run(max_tasks=5)`	`max_tasks: int`	`{tasks_processed, tasks_failed}`
`_generate_ad_copy(context)`	`{target_audience, pain_point, offer, url}`	`{ad_copy, status}`
`_generate_email_sequence(context)`	`{prospect_name, company, pain_point}`	`{email_1, email_2, email_3, status}`
`_generate_linkedin_post(context)`	`{topic}`	`{post, status}`
`_generate_sms(context)`	`{name}`	`{sms, status}`

Job types handled: `content_ad_copy`, `content_email`, `content_linkedin`, `content_sms`

System prompt: Australian B2B copywriter, plain and punchy, no corporate speak, Australian English spelling.

---

analytics_agent.py — Analytics Head (Tier 2)

Purpose: Daily analytics cycle. Analyses leads and experiments, tracks budget burn rate, and checks circuit breakers.

Scheduler trigger: Daily 09:00 UTC (as part of `_run_department_heads`)

Key functions:

FUNCTION	PARAMS	RETURNS
`run()`	—	Full analytics summary dict
`_analyse_leads()`	—	`{total_7d, qualified, hot_leads, converted, conversion_rate, avg_score}`
`_analyse_experiments()`	—	`{total_7d, complete, killed, running, success_rate, total_revenue_aud, roi}`
`_analyse_budget()`	—	`{weekly_budget_aud, used_aud, remaining_aud, burn_rate, on_track}`
`_check_circuit_breakers(budget_stats)`	`dict`	Circuit breaker result
`get_conversion_stats()`	—	Full lead pipeline breakdown

What it reads: `leads` (last 7 days), `experiments` (last 7 days), `circuit_breaker_log`

What it writes: `cold_ledger` (ANALYTICS_CYCLE event), may trigger `circuit_breaker_log` entries

worker.py — Tier 3 Worker

Purpose: Stateless task executor. Picks up one task from the queue, executes it, posts a pheromone signal, and terminates. All state is in the database.

Trigger: Called directly by department heads, or via `python worker.py [task_id]`

Key functions:

FUNCTION	PARAMS	RETURNS
`run_task(task_id=None)`	`task_id: int \`	None`	`{status, task_id, result}`
`_execute(job_type, context)`	`str, dict`	Result dict with `signal_type`, `topic`, `strength`

Job types handled:

JOB TYPE	HANDLER
`research_prospect`	`solar_research_agent.research()`
`research_market`	Placeholder market analysis
`content_ad_copy`	`content_agent._generate_ad_copy()`
`content_email`	`content_agent._generate_email_sequence()`
`content_linkedin`	`content_agent._generate_linkedin_post()`
`content_sms`	`content_agent._generate_sms()`
`solar_qualify_lead`	`qualification_agent.qualify()`
`solar_research_company`	`solar_research_agent.research()`
`solar_generate_proposal`	`proposal_agent.generate()`
`generic_test`	Returns success dict

Pheromone signal strength by outcome:

- Successful prospect (score ≥ 6) → `POSITIVE`, strength = score/10
- Failed task → `NEGATIVE`, strength = 0.3
- Neutral tasks → `NEUTRAL`, strength = 0.5

scout_agent.py — Prospect Hunter

Purpose: Daily autonomous discovery of Australian solar companies likely to need CRM automation. Fuses three data sources, scores candidates, queues top 10 for deep research, and saves top 5 as opportunities.

Scheduler trigger: Daily 08:00 UTC

Key functions:

FUNCTION	PARAMS	RETURNS
`run()`	—	`{prospects_found, queued_for_research, opportunities_saved}`
`_gather_candidates()`	—	Raw candidates from all sources
`_score_candidate(candidate)`	`dict`	Candidate with `scout_score` 0–10
`_queue_for_research(prospects)`	`list`	Count queued to message bus
`_save_opportunities(prospects)`	`list`	Count saved to opportunities table

Scoring logic:

SOURCE	BASE SCORE	HOT SIGNAL BONUS
`installer_registry`	3.0	+1.5 per hot keyword hit
`social_signal`	5.0	+2.0 if "hiring" present
`knowledge_graph`	4.0	+1.5 per hot keyword hit

Hot signals: hiring, scaling, crm, manual process, follow up, new office

If the company already appears in the `leads` table, `-3.0` is applied to avoid double-handling.

What it reads: `collected_data` (last 2 days), `kg_entities`, `leads`

What it writes: `message_bus` (TASK to research_queue), `opportunities` table

---

mutation_engine.py — Strategy Evolution

Purpose: Monday retrospective job. Finds underperforming experiments (score < 5 or 3+ consecutive failures), generates 2 mutated variants per loser using GPT-4o, and either submits them as new experiments or kills the parent.

Scheduler trigger: Monday 22:30 UTC

Key functions:

FUNCTION	PARAMS	RETURNS
`run()`	—	`{analysed, mutations_created, killed}`
`_find_underperformers()`	—	Up to 5 experiments with score < 5 or consecutive_failures ≥ 3
`_mutate(experiment)`	`dict`	`{mutations, kill_recommendation, kill_reason}`
`_submit_mutations(mutations, parent)`	`list, dict`	Inserts new experiment rows with status `queued`
`_kill_experiment(experiment)`	`dict`	Updates experiment status to `killed`

Mutation output format from GPT-4o:

```
{
  "mutations": [{
    "name": "Experiment v2 — Faster Follow-up",
    "hypothesis": "Calling within 5 minutes increases conversion",
    "changes_from_parent": ["Reduce SMS-to-call delay from 30min to 5min"],
    "expected_improvement": "Industry data shows 5min = 9x higher connect rate",
    "confidence": 0.72,
    "estimated_revenue": 3000,
    "risk_level": "low"
  }],
  "kill_recommendation": false,
  "kill_reason": ""
}
```

ab_tester.py — A/B Test Manager

Purpose: Creates, tracks, and evaluates A/B tests. Declares winners when statistical thresholds are met and emits pheromone signals to reinforce winning strategies.

Scheduler trigger: Daily 10:00 UTC (`evaluate_tests()`)

Key functions:

FUNCTION	PARAMS	RETURNS
`create_test(name, hypothesis, variant_a, variant_b, metric, experiment_id)`	Various	`test_id: str`
`record_event(test_id, variant, event_type, value=1.0)`	Various	None
`evaluate_tests()`	—	`{evaluated, winners_found, tests_extended}`
`get_summary()`	—	Status counts dict

Evaluation rules:

- Minimum 30 impressions per variant before evaluation
- Less than 10% lift → `no_winner`
- ≥ 10% lift → winner is variant with higher conversion rate
- Winner emits `ab_winner` pheromone signal at strength 0.8

Metric options: `conversion_rate`, `reply_rate`, `booking_rate`

What it reads/writes: `ab_tests` table

---

qualification_agent.py — Lead Scorer

Purpose: Scores solar leads 1–10 using GPT-4o. Triggers outbound Retell calls for hot leads (score ≥ 7 with phone number). Sends Slack alerts.

Trigger: Called by `ghl_handler.py` on each new webhook lead, or as worker task `solar_qualify_lead`

Key functions:

FUNCTION	PARAMS	RETURNS	
`qualify(lead_data, lead_id=None)`	`dict, int \`	None`	`{score, reason, recommended_action, key_signals, risk_flags}`
`_ai_score(lead_data)`	`dict`	GPT-4o scoring result	
`_rule_based_score(lead_data)`	`dict`	Rule-based fallback scoring	
`_trigger_outbound_call(lead_data, lead_id)`	`dict, int`	Fires Retell call if configured	

Scoring criteria (rule-based fallback):

SIGNAL	POINTS
Homeowner (owner)	+3
Bill > $300/mo	+3
Bill $200–300/mo	+2
Bill $150–200/mo	+1
Ideal roof (tile/colorbond < 15yr)	+2
High sun state (QLD/WA/SA/NSW/NT)	+2

Routing: score ≥ 7 → `call_now`, 5–6 → `nurture`, ≤ 4 → `disqualify`

What it writes: `leads` (qualification_score, score_reason, recommended_action, status), `cold_ledger`, `call_logs` (if outbound call initiated)

---

proposal_agent.py — Proposal Generator

Purpose: Generates 3-section business proposals for solar company prospects and saves them to `proposals/`.

Trigger: Worker task `solar_generate_proposal`, or called directly via CLI

Key functions:

FUNCTION	PARAMS	RETURNS
`generate(client_name, pain_points, current_process, estimated_leads_per_month)`	Various	`{file_path, client_name, proposal_text, generated_at}`

Proposal structure:

1. Current State — problem quantified with estimated cost of lost leads (25% × avg $8,000–12,000 per system)
2. Future State — what the AI automation system delivers, with specific metrics
3. Investment — three tiers: Starter $1,500/mo, Growth $2,000/mo (recommended), Premium $2,500/mo

Output: Saved as `proposals/{ClientName}_{YYYYMMDD}.txt`

---

solar_research_agent.py — Company Profiler

Purpose: Given a solar company name and suburb, generates a structured research profile for personalised outreach.

Trigger: Research task `research_prospect`, or called directly

Key functions:

FUNCTION	PARAMS	RETURNS
`research(company_name, suburb)`	`str, str`	Full research profile dict
`batch_research(companies)`	`list[{company_name, suburb}]`	List sorted by score DESC
`get_outreach_message(research_result, sender_name)`	`dict, str`	Personalised email/LinkedIn message

Research profile fields: company_name, suburb, website, google_maps_present, estimated_staff, owner_name, review_summary, admin_pain_points, crm_signals (has_booking_link, has_contact_form, has_chat_widget), outreach_angle, score (1–10)

---

report_agent.py — Client Report Generator

Purpose: Generates weekly performance reports for solar clients and saves them to `reports/`.

Trigger: Called directly via CLI or scheduled via `generate_all_clients()`

Key functions:

FUNCTION	PARAMS	RETURNS
`generate(client_name, date_range=7)`	`str, int`	`{file_path, report_text, stats, generated_at}`
`generate_all_clients(days=7)`	`int`	List of report results

Stats collected: total_leads, qualified (≥5/10), hot_leads (≥7/10), contacted, converted, disqualified, avg_score, contact_rate_pct, conversion_rate_pct, leads_not_lost (estimated 25%), estimated_revenue_protected (leads_not_lost × $8,000)

Output: Saved as `reports/{ClientName}_report_{YYYYMMDD}.txt`

---

red_team_agent.py — Devil's Advocate

Purpose: Analyses experiment ideas for failure modes. Returns a `devil_score` (1–10, higher = more flaws) that triggers automatic confidence downgrades in the General.

Trigger: Called by `master_agent._process_idea()` for every experiment idea

Key functions:

FUNCTION	PARAMS	RETURNS	
`analyse(idea_text, experiment_id=None)`	`str, int \`	None`	`{devil_score, failure_modes, summary}`
`adjust_confidence(confidence_score, devil_score)`	`float, int`	Adjusted confidence score	

Confidence adjustment formula:

- If `devil_score > 6`: `penalty = (devil_score − 6) × 0.5`, `adjusted = confidence − penalty`
- Example: confidence 7.5, devil_score 8 → penalty 1.0 → adjusted 6.5

Failure modes returned (3 per analysis): ranked by severity, each with mode title and 2-sentence explanation

What it writes: `cold_ledger` (RED_TEAM_ANALYSIS event)